

Lab-4.1

Xây dựng ANN với Tensorflow

```
[ ]: !pip install tensorflow==2.12.0
```

The most basic steps to build ANN - Các bước xây dựng ANN căn bản nhất

- Data Preprocessing - Tiền xử lý dữ liệu
- Add input layer - Thêm lớp đầu vào
- Random w init - Khởi tạo w ngẫu nhiên
- Add Hidden Layers - Thêm lớp ẩn
- Select Optimizer, Loss, and Performance Metrics - Chọn số liệu tối ưu hóa, mất mát và hiệu suất
- Compile the model - Biên dịch mô hình
- use model.fit to train the model - sử dụng model.fit để đào tạo mô hình
- Evaluate the model - Đánh giá mô hình
- Adjust optimization parameters or model if needed - Điều chỉnh các tham số tối ưu hóa hoặc mô hình nếu cần

```
[3]: import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Flatten, Dense
```

```
[4]: print(tf.__version__)
```

2.12.0

```
[5]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
[6]: dataset = pd.read_csv('/content/Customer_Churn_Modelling.csv')
```

```
[7]: dataset.head()
```

```
[7]:
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	\
0	1	15634602	Hargrave	619	France	Female	42	
1	2	15647311	Hill	608	Spain	Female	41	
2	3	15619304	Onio	502	France	Female	42	
3	4	15701354	Boni	699	France	Female	39	
4	5	15737888	Mitchell	850	Spain	Female	43	

	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	\
0	2	0.00	1	1	1	
1	1	83807.86	1	0	1	
2	8	159660.80	3	1	0	

3	1	0.00	2	0	0
4	2	125510.82	1	1	1

	EstimatedSalary	Exited
0	101348.88	1
1	112542.58	0
2	113931.57	1
3	93826.63	0
4	79084.10	0

```
[8]: X = dataset.drop(labels=['CustomerId', 'Surname', 'RowNumber', 'Exited'], axis=1)
      y = dataset['Exited']
```

```
[9]: X.head()
```

```
[9]:   CreditScore  Geography  Gender  Age  Tenure  Balance  NumOfProducts  \
0          619    France  Female   42      2      0.00              1
1          608     Spain  Female   41      1  83807.86              1
2          502    France  Female   42      8 159660.80              3
3          699    France  Female   39      1      0.00              2
4          850     Spain  Female   43      2 125510.82              1
```

	HasCrCard	IsActiveMember	EstimatedSalary
0	1	1	101348.88
1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10

```
[10]: y.head()
```

```
[10]: 0    1
      1    0
      2    1
      3    0
      4    0
      Name: Exited, dtype: int64
```

```
[11]: from sklearn.preprocessing import LabelEncoder
```

```
[12]: label1 = LabelEncoder()
      X['Geography'] = label1.fit_transform(X['Geography'])
```

```
[13]: X.head()
```

```
[13]:
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	\
0	619	0	Female	42	2	0.00	1	
1	608	2	Female	41	1	83807.86	1	
2	502	0	Female	42	8	159660.80	3	
3	699	0	Female	39	1	0.00	2	
4	850	2	Female	43	2	125510.82	1	

	HasCrCard	IsActiveMember	EstimatedSalary
0	1	1	101348.88
1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10

```
[14]: label = LabelEncoder()
X['Gender'] = label.fit_transform(X['Gender'])
X.head()
```

```
[14]:
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	\
0	619	0	0	42	2	0.00	1	
1	608	2	0	41	1	83807.86	1	
2	502	0	0	42	8	159660.80	3	
3	699	0	0	39	1	0.00	2	
4	850	2	0	43	2	125510.82	1	

	HasCrCard	IsActiveMember	EstimatedSalary
0	1	1	101348.88
1	0	1	112542.58
2	1	0	113931.57
3	0	0	93826.63
4	1	1	79084.10

```
[15]: X = pd.get_dummies(X, drop_first=True, columns=['Geography'])
X.head()
```

```
[15]:
```

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	\
0	619	0	42	2	0.00	1	1	
1	608	0	41	1	83807.86	1	0	
2	502	0	42	8	159660.80	3	1	
3	699	0	39	1	0.00	2	0	
4	850	0	43	2	125510.82	1	1	

	IsActiveMember	EstimatedSalary	Geography_1	Geography_2
0	1	101348.88	False	False
1	1	112542.58	False	True
2	0	113931.57	False	False
3	0	93826.63	False	False

4 1 79084.10 False True

0.0.1 Feature Standardization

```
[16]: from sklearn.preprocessing import StandardScaler
```

```
[17]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
↳ random_state = 0, stratify = y)
```

```
[18]: scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
[19]: X_train
```

```
[19]: array([[ -1.24021723, -1.09665089,  0.77986083, ...,  1.64099027,
        -0.57812007, -0.57504086],
        [ 0.75974873,  0.91186722, -0.27382717, ..., -1.55587522,
        1.72974448, -0.57504086],
        [-1.72725557, -1.09665089, -0.9443559 , ...,  1.1038111 ,
        -0.57812007, -0.57504086],
        ...,
        [-0.51484098,  0.91186722,  0.87565065, ..., -1.01507508,
        1.72974448, -0.57504086],
        [ 0.73902369, -1.09665089, -0.36961699, ..., -1.47887193,
        -0.57812007, -0.57504086],
        [ 0.95663657,  0.91186722, -1.32751517, ...,  0.50945854,
        -0.57812007,  1.73900686]])
```

0.0.2 Build ANN

```
[20]: model = Sequential()
model.add(Dense(X.shape[1], activation='relu', input_dim = X.shape[1]))
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation = 'sigmoid'))
```

```
[21]: X.shape[1]
```

```
[21]: 11
```

```
[22]: model.compile(optimizer='adam', loss = 'binary_crossentropy',
↳ metrics=['accuracy'])
```

```
[23]: model.fit(X_train, y_train.to_numpy(), batch_size = 10, epochs = 10, verbose =
↳ 1)
```

Epoch 1/10

800/800 [=====] - 4s 4ms/step - loss: 0.4551 -

```

accuracy: 0.8035
Epoch 2/10
800/800 [=====] - 2s 3ms/step - loss: 0.4127 -
accuracy: 0.8267
Epoch 3/10
800/800 [=====] - 2s 3ms/step - loss: 0.3779 -
accuracy: 0.8451
Epoch 4/10
800/800 [=====] - 3s 4ms/step - loss: 0.3534 -
accuracy: 0.8574
Epoch 5/10
800/800 [=====] - 3s 3ms/step - loss: 0.3449 -
accuracy: 0.8599
Epoch 6/10
800/800 [=====] - 5s 6ms/step - loss: 0.3411 -
accuracy: 0.8583
Epoch 7/10
800/800 [=====] - 3s 4ms/step - loss: 0.3370 -
accuracy: 0.8634
Epoch 8/10
800/800 [=====] - 3s 4ms/step - loss: 0.3357 -
accuracy: 0.8621
Epoch 9/10
800/800 [=====] - 2s 3ms/step - loss: 0.3338 -
accuracy: 0.8618
Epoch 10/10
800/800 [=====] - 3s 4ms/step - loss: 0.3328 -
accuracy: 0.8619

```

[23]: <keras.callbacks.History at 0x7fb5ce351a80>

```

[25]: y_pred = model.predict(X_test)

# Convert the predictions to class labels
# (assuming binary classification with a sigmoid activation in the output layer)
y_pred = (y_pred > 0.5).astype("int32")

63/63 [=====] - 0s 976us/step

```

[26]: y_pred

```

[26]: array([[0],
            [0],
            [0],
            ...,
            [0],
            [1],
            [0]], dtype=int32)

```

```
[27]: y_test
```

```
[27]: 1344    1
      8167    0
      4747    0
      5004    1
      3124    1
      ..
      9107    0
      8249    0
      8337    0
      6279    1
      412     0
      Name: Exited, Length: 2000, dtype: int64
```

```
[28]: model.evaluate(X_test, y_test.to_numpy())
```

```
63/63 [=====] - 0s 1ms/step - loss: 0.3512 - accuracy: 0.8560
```

```
[28]: [0.35123834013938904, 0.8560000061988831]
```

```
[29]: from sklearn.metrics import confusion_matrix, accuracy_score
```

```
[30]: confusion_matrix(y_test, y_pred)
```

```
[30]: array([[1545,   48],
          [ 240,  167]])
```

```
[31]: accuracy_score(y_test, y_pred)
```

```
[31]: 0.856
```

Thank you for your attention!